

Ejercicio 1: Simulación de Ecosistema

La aplicación que se pide representa un ecosistema animal, mediante una arquitectura de interfaces, clases y relaciones entre ellas. Las especificaciones son las siguientes:

1. Interfaces:
 - Depredador: método boolean cazar(Animal presa)
 - Presa: método void huir()
2. Clase base Animal: se caracteriza por su *energía* (oscila entre 1-100. Al crearse tendrá valor 1) y su *nombre*.
 - Método comer(int cantidad): recupera energía
3. Implementaciones:
 - Leon: Es Animal y Depredador.
 - Conejo: Es Animal y Presa.

Los leones, cuando comen recuperan su energía en un valor que es igual a la cantidad que han comido (sin superar el máximo).

Los conejos al comer recuperan la mitad de la cantidad comida (sin superar el máximo).

Las presas al huir pierden su energía por completo (energía a 1).

Cuando un depredador caza a un Animal, el resultado de la caza será true o false.

- TRUE: si el Animal es una Presa y si la energía del cazador es mayor o igual a la energía de la presa. En este caso el cazador caza la presa y se la come (la cantidad comida es igual al valor de la energía de la presa)
- FALSE: en cualquier otro caso, en este caso la presa huye y el cazador pierde la mitad de su energía

4. Programa principal

- Crea una lista de animales con estos animales de prueba:
Conejo, León, Conejo, Conejo, León, Conejo.
(Nota: el programa tiene que funcionar con cualquier lista de animales, no solo con esta).
- Recórrela de tal forma que todos los animales coman una cantidad aleatoria entre 1 y 60
- Recórrela hasta encontrar al primer depredador, y haz que cace a todas las presas de la lista, mostrando los éxitos y fracasos de las cazas.
 - Si el cazador consigue cazar a una presa, se elimina ese animal de la lista.
 - Si el cazador no consigue cazar a ninguna presa, se elimina de la lista.
- Imprime la lista que ha quedado (nombre y energía)

Ejercicio 2: Gestor de Tareas de CPU

Simula un planificador de procesos de un Sistema Operativo.

1. Clase Proceso: pid (int), prioridad (int, 0=máxima, 10=mínima), duración (int)

La propiedad pid la genera automáticamente la aplicación para asegurar unicidad

2. Los procesos implementarán un orden por prioridad, y en caso de empate, por duración (menor duración primero - *Shortest Job First*).

3. Clase Planificador:

- Tiene una lista de procesos pendientes y un conjunto con los procesos que se están ejecutando
- Método agregarProcesoPendiente(int prioridad, int duracion): los procesos se irán insertando según su orden (se puede usar el método sort)
- Método ejecutar(): Extrae un proceso de la lista, de tal forma que se extrae siempre el de mayor prioridad y simula su ejecución imprimiendo un mensaje. El proceso pasa al conjunto de procesos en ejecución
- Método listarEjecucion(): devuelve una lista de los procesos que se están ejecutando ordenados por pid (se puede usar el método sort)
- Método abortarProceso(int pid): elimina del conjunto de procesos en ejecución el proceso cuyo pid se pasa como parámetro

en ejecucion:[]

en ejecucion:[[pid=1, pri=4, dur=54], [pid=5, pri=3, dur=4], [pid=7, pri=7, dur=2], [pid=9, pri=0, dur=44]]

en ejecucion:[[pid=5, pri=3, dur=4], [pid=7, pri=7, dur=2], [pid=9, pri=0, dur=44]]

	Ejer1	Ejer2
RA2 (Objetos y clases)	5 puntos: clase Animal, implementación subclases	5 puntos: clase Proceso y planificador
RA4y RA7 (Herencia, Polimorfismo, Clases Abstractas, Interfaces)	10 puntos: upcasting y downcasting, uso de polimorfismo, reconocimiento de instancias	
RA6 (Colecciones)	3 puntos: uso correcto de Colecciones.	7: uso correcto de colecciones